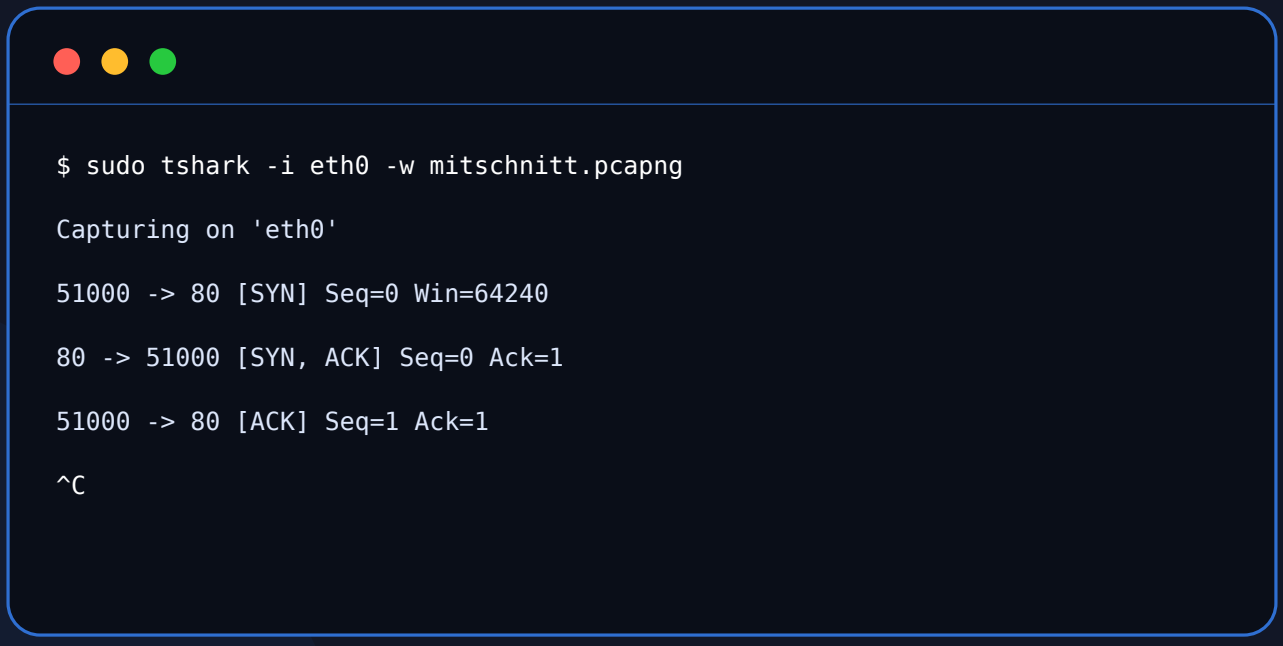
PRAXISHANDBUCH

Wireshark

Das Praxishandbuch

Netzwerkverkehr sichtbar machen – vom ersten Mitschnitt bis zur Protokollanalyse



MITSCHNEIDEN STATT ERZEUGEN. NUR IM EIGENEN NETZ VERSTEHEN.

Mit Übungen, Spickzettel, Glossar und Index · Ausgabe Oktober 2026

Wireshark
Das Praxishandbuch

Netzwerkverkehr sichtbar machen – vom ersten Mitschnitt bis zur Protokollanalyse

Von Capture- und Display-Filtern über Protokollanalyse bis zu Statistiken und tshark.
Mit geführten Übungen in einem eigenen, isolierten Testnetz, Spickzettel, Glossar und Index.

Inhaltsverzeichnis

Vorwort 5

Für wen ist dieses Handbuch? 5

Aufbau 6

Kapitel 1: Was ist Wireshark? 7

1.1 Einordnung: mitschneiden statt erzeugen 7

1.2 Typische Einsatzgebiete 7

1.3 Der grundsätzliche Analyseablauf 8

1.4 Passiv ist nicht automatisch unproblematisch 8

Zusammenfassung 8

Kapitel 2: Grundlagen – Schnittstellen, Promiscuous Mode, OSI-Modell 9

2.1 Netzwerkschnittstellen 9

2.2 Promiscuous Mode 9

2.3 Das OSI-Modell als Lesehilfe 10

Zusammenfassung 10

Kapitel 3: Installation und Laborumgebung 11

3.1 Installation 11

3.2 Rechte für Mitschnitte ohne root 11

3.3 Grafische Oberfläche und Kommandozeile 12

3.4 Das isolierte Testnetz 12

Zusammenfassung 12

Kapitel 4: Die Oberfläche – Paketliste, Paketdetails, Bytes-Ansicht 13

4.1 Einen Mitschnitt starten 13

4.2 Die drei zentralen Fensterbereiche 13

4.3 Zwischen den Ansichten navigieren 13

4.4 Farbregeln 14

Zusammenfassung 14

Kapitel 5: Capture-Filter vs. Display-Filter 15

5.1 Der entscheidende Unterschied 15

5.2 Capture-Filter (BPF-Syntax) 15

5.3 Display-Filter (Wireshark-Syntax) 16

5.4 Filter kombinieren 16

Zusammenfassung . . . 16

Kapitel 6: Protokolle erkennen – TCP, HTTP, DNS, ARP . . . 17

6.1 Den TCP-Drei-Wege-Handshake wiedererkennen . . . 17
6.2 HTTP-Anfragen und -Antworten . . . 17
6.3 DNS-Auflösung beobachten . . . 18
6.4 ARP: Adressauflösung im lokalen Netz . . . 18
Zusammenfassung . . . 18

Kapitel 7: TCP-Streams folgen und Klartext-Risiken erkennen . . . 19

7.1 Eine ganze Verbindung zusammenhängend lesen . . . 19
7.2 Klartext-Zugangsdaten in einem eigenen Testaufbau . . . 19
7.3 Warum Verschlüsselung den entscheidenden Unterschied macht . . . 20
Zusammenfassung . . . 20

Kapitel 8: Statistiken und Visualisierung . . . 21

8.1 Protocol Hierarchy: Was steckt überhaupt im Mitschnitt? . . . 21
8.2 Conversations: Wer spricht mit wem? . . . 21
8.3 IO-Graph: Verkehr über die Zeit . . . 21
Zusammenfassung . . . 22

Kapitel 9: Mitschnitte speichern, exportieren und mit tshark automatisieren . . . 23

9.1 Speicherformate . . . 23
9.2 Objekte exportieren . . . 23
9.3 Automatisierung mit tshark . . . 23
Zusammenfassung . . . 24

Kapitel 10: Praxisteil – Laboraufbau und geführte Übungen . . . 25

10.1 Laboraufbau . . . 25
10.2 Checkliste: sichere Laborumgebung . . . 25
10.3 Übungen . . . 25
Zusammenfassung . . . 27

Kapitel 11: Troubleshooting, Spickzettel, Glossar und Ressourcen . . . 28

11.1 Häufige Probleme . . . 28
11.2 Spickzettel . . . 28
11.3 Glossar . . . 29
11.4 Weiterführende Ressourcen . . . 29
Zusammenfassung . . . 29

Verzeichnis der Befehlsbeispiele **30**

Index **31**

Vorwort

`Wireshark` ist der Gegenpol zu den anderen beiden Werkzeugen dieser Reihe: Während `hping3` Pakete von Hand erzeugt und `Nmap` automatisiert Pakete verschickt und auswertet, erzeugt Wireshark **gar nichts** – es schneidet Netzwerkverkehr mit und macht ihn sichtbar, bis hinunter zum einzelnen Bit. Wer verstehen will, was ein mit `hping3` gebautes Paket tatsächlich bewirkt oder was ein `Nmap`-Scan auf der Leitung tatsächlich erzeugt, findet in Wireshark das passende Werkzeug, um genau hinzusehen.

Dieses Handbuch ergänzt das Ethical-Hacking-, das hping3- und das Nmap-Handbuch derselben Reihe um die Analyseseite: Wie liest man einen Mitschnitt, wie filtert man gezielt, und wie erkennt man in echtem Verkehr das wieder, was man in den anderen Büchern bereits als Theorie kennengelernt hat?

> **ILLEGAL – niemals ohne Auftrag**
>
> Wireshark **erzeugt** keine Pakete, sondern **liest mit**. Genau das macht den rechtlichen Rahmen anders, aber nicht milder: Das unbefugte Abfangen fremder, nicht für dich bestimmter Datenübermittlung ist nach § 202b StGB (Abfangen von Daten) strafbar – unabhängig davon, ob du die Daten anschließend nutzt. Schneide **ausschließlich** Verkehr mit, der dir selbst gehört, für den du eine ausdrückliche Einwilligung besitzt (z. B. als Administrator:in eines Netzwerks mit entsprechender Dokumentation), oder der in deiner eigenen, isolierten Laborumgebung entsteht.

Für wen ist dieses Handbuch?

Du bist hier richtig, wenn du

- bereits das hping3- oder Nmap-Handbuch kennst und verstehen willst, wie sich deren Techniken tatsächlich auf der Leitung zeigen,
- Netzwerkprobleme in deinem eigenen Netz systematisch diagnostizieren möchtest,
- verstehen willst, warum unverschlüsselte Protokolle ein echtes Risiko sind, statt es nur zu behaupten,
- dich auf Zertifizierungen vorbereitest, in denen Paketanalyse eine Rolle spielt (eJPT, OSCP, CompTIA Network+/Security+).

Aufbau

Kapitel	Thema
1	Was ist Wireshark? Einordnung, Geschichte, Abgrenzung zu hping3/Nmap
2	Grundlagen: Netzwerkschnittstellen, Promiscuous Mode, das OSI-Modell in der Praxis
3	Installation und Laborumgebung
4	Die Oberfläche: Paketliste, Paketdetails, Bytes-Ansicht
5	Capture-Filter vs. Display-Filter
6	Protokolle erkennen: TCP, HTTP, DNS, ARP
7	TCP-Streams folgen und Klartext-Risiken erkennen
8	Statistiken und Visualisierung
9	Mitschnitte speichern, exportieren und mit tshark automatisieren
10	Praxisteil: Laboraufbau und geführte Übungen
11	Troubleshooting, Spickzettel, Glossar und Ressourcen

> **TIPP – Profi-Trick**
>
> Die Kombination aus allen drei Büchern ist besonders lehrreich: Baue mit `hping3` ein Paket, beobachte es live in Wireshark, und vergleiche danach, wie `Nmap` dieselbe Aufgabe automatisiert löst. Kapitel 6 greift das an mehreren Stellen gezielt auf.

Kapitel 1: Was ist Wireshark?

Lernziele

- Du kannst Wireshark funktional von hping3 und Nmap abgrenzen.
- Du kennst die Geschichte und die wichtigsten Einsatzgebiete von Wireshark.
- Du verstehst den grundsätzlichen Ablauf einer Paketanalyse.

1.1 Einordnung: mitschneiden statt erzeugen

`hping3` baut einzelne Pakete von Hand, `Nmap` automatisiert ganze Scan-Abläufe – beide Werkzeuge **erzeugen aktiv** Netzwerkverkehr. Wireshark tut das Gegenteil: Es **liest passiv mit**, was ohnehin über eine Netzwerkschnittstelle läuft, und stellt es in einer durchsuchbaren, hierarchisch aufgebauten Form dar.

Werkzeug	Grundidee	Verhältnis zum Netzwerkverkehr
`hping3`	manuell konstruierte Einzelpakete	erzeugt aktiv
`Nmap`	automatisierte Scan-Logik	erzeugt aktiv, in großem Umfang
`Wireshark`	Mitschnitt und Analyse	liest passiv mit, erzeugt selbst nichts

Gut zu wissen

Wireshark entstand 1998 unter dem Namen **Ethereal**, entwickelt von Gerald Combs. 2006 wurde das Projekt aus rechtlichen Gründen rund um den Markennamen in **Wireshark** umbenannt; der Code und die Community blieben identisch. Heute ist es der De-facto-Standard für Paketanalyse, sowohl in der Fehlerdiagnose als auch in der Sicherheitsforschung.

1.2 Typische Einsatzgebiete

- **Netzwerk-Troubleshooting**: Warum ist eine Verbindung langsam, bricht ab, oder kommt gar nicht erst zustande?
- **Protokollanalyse und Lernen**: Den Drei-Wege-Handshake, DNS-Auflösung oder TLS-Aushandlung nicht nur in der Theorie, sondern am echten Mitschnitt nachvollziehen.
- **Sicherheitsanalyse**: Erkennen von Klartext-Zugangsdaten, verdächtigem Verkehr oder den Spuren eines Scans (Kapitel 6 und 7).
- **Verifikation eigener Werkzeuge**: Genau sehen, was ein mit hping3 gebautes Paket oder ein Nmap-Scan tatsächlich auf die Leitung bringt.
- **Forensik**: Nachträgliche Analyse gespeicherter Mitschnitte nach einem Sicherheitsvorfall.

1.3 Der grundsätzliche Analyseablauf

Beispiel 1.1: Typischer Ablauf einer Paketanalyse (Konzept)

```
1. Netzwerkschnittstelle auswählen (Kapitel 2)
2. Mitschnitt starten, optional mit Capture-Filter eingrenzen (Kapitel 5)
3. Verkehr erzeugen oder abwarten (z. B. eine Webseite aufrufen)
4. Mitschnitt stoppen
5. Mit Display-Filtern gezielt die relevanten Pakete finden (Kapitel 5)
6. Einzelne Pakete und ganze Streams im Detail untersuchen (Kapitel 6-7)
7. Ergebnis sichern oder exportieren (Kapitel 9)
```

1.4 Passiv ist nicht automatisch unproblematisch

LEGAL – nur mit Auftrag/Einwilligung

Dass Wireshark nichts „angreift", sondern nur mitliest, lässt die rechtliche Lage leicht harmloser wirken, als sie ist. Mitgeschnittener Verkehr enthält häufig personenbezogene Daten, Zugangsdaten oder vertrauliche Inhalte Dritter – unabhängig von deiner Absicht. Die Autorisierung entscheidet auch hier: dein eigenes Netz, eine dokumentierte administrative Berechtigung, oder eine vollständig isolierte Laborumgebung (Kapitel 3 und 10).

Zusammenfassung

Zusammenfassung

- Wireshark liest passiv mit, statt wie hping3 und Nmap aktiv Verkehr zu erzeugen.
- Haupteinsatzgebiete sind Troubleshooting, Protokollanalyse, Sicherheitsanalyse, Werkzeug-Verifikation und Forensik.
- Eine vollständige Analyse durchläuft typischerweise sieben Schritte von der Schnittstellenwahl bis zum Export.
- Passives Mitschneiben ist rechtlich nicht automatisch unkritisch; auch hier entscheidet die Autorisierung.

Kapitel 2: Grundlagen – Schnittstellen, Promiscuous Mode, OSI-Modell

Lernziele

- Du verstehst, warum die Wahl der richtigen Netzwerkschnittstelle entscheidend ist.
- Du kennst den Unterschied zwischen normalem und Promiscuous Mode.
- Du kannst das OSI-Modell nutzen, um einen Mitschnitt systematisch zu lesen.

2.1 Netzwerkschnittstellen

Wireshark schneidet immer an einer bestimmten **Netzwerkschnittstelle** (Interface) mit – einer Ethernet-Karte, einem WLAN-Adapter, einer virtuellen Maschinen-Schnittstelle oder einem Loopback-Interface.

Beispiel 2.1: Verfügbare Schnittstellen auflisten

```
$ tshark -D
1. eth0
2. lo (Loopback)
3. any (alle Schnittstellen gleichzeitig)
```

Gut zu wissen

Die Schnittstelle `any` ist praktisch zum schnellen Ausprobieren, zeigt aber nicht immer dieselben Details wie eine einzelne, konkrete Schnittstelle (z. B. fehlen bei `any` unter Linux oft die Ethernet-Header). Für genaue Analysen wählst du besser gezielt die tatsächlich relevante Schnittstelle.

2.2 Promiscuous Mode

Im Normalbetrieb verwirft eine Netzwerkkarte Pakete, die nicht an ihre eigene Adresse gerichtet sind. Im **Promiscuous Mode** nimmt sie dagegen jedes Paket an, das physisch auf dem Medium ankommt, unabhängig von der Zieladresse.

Beispiel 2.2: Promiscuous Mode beim Start eines Mitschnitts

```
$ sudo tshark -i eth0 -p   # -p deaktiviert Promiscuous Mode (nur eigene Pakete)
$ sudo tshark -i eth0      # Promiscuous Mode ist in der Voreinstellung aktiv
```

WARNUNG – Sicherheitsrisiko

Promiscuous Mode sorgt dafür, dass auch Pakete erfasst werden, die **nicht an dich gerichtet sind**. In einem gemeinsam genutzten Netz (klassisches Hub-Netz, manche WLAN-Konfigurationen) kann das fremden Verkehr einschließen, den mitzuschneiden eine eigenständige Straftat nach § 202b StGB sein kann (siehe Vorwort). In modernen, geswitchten Netzen siehst du dadurch in der Praxis trotzdem meist nur Verkehr, der für dein Segment bestimmt ist – verlasse dich aber nicht darauf, sondern auf die Autorisierung.

2.3 Das OSI-Modell als Lesehilfe

Wireshark stellt jedes Paket **geschichtet** dar, in derselben Reihenfolge wie das OSI-Modell:

OSI-Schicht	Beispielprotokolle	In Wireshark sichtbar als
2 – Sicherung	Ethernet, ARP	„Ethernet II"
3 – Vermittlung	IP, ICMP	„Internet Protocol Version 4"
4 – Transport	TCP, UDP	„Transmission Control Protocol"
5–7 – Sitzung/Darstellung/Anwendung	HTTP, DNS, TLS	z. B. „Hypertext Transfer Protocol"

Beispiel 2.3: Die Schichten eines einzelnen Pakets (Konzept)

```
Frame 42: 74 bytes on wire
  Ethernet II, Src: ..., Dst: ...
    Internet Protocol Version 4, Src: 192.168.56.101, Dst: 192.168.56.102
      Transmission Control Protocol, Src Port: 51000, Dst Port: 80
        Hypertext Transfer Protocol
```

TIPP – Profi-Trick

Wenn du ein Feld in der obersten, zusammengefassten Zeile eines Pakets nicht verstehst, klapp in der Paketdetailansicht (Kapitel 4) die passende Schicht auf – fast jedes Feld, das dir aus dem hping3-Handbuch (Kapitel 2) bekannt ist (TTL, Flags, Fenstergröße), findest du hier als eigene, benannte Zeile wieder.

Zusammenfassung

Zusammenfassung

- Wireshark schneidet an einer konkreten Netzwerkschnittstelle mit; `any` ist praktisch, aber weniger detailliert.
- Promiscuous Mode erfasst auch nicht an dich gerichtete Pakete – mit entsprechender rechtlicher Vorsicht.
- Jedes Paket wird in OSI-Schichten dargestellt, von Ethernet über IP und TCP bis zur Anwendungsschicht.
- Bekannte Felder aus dem hping3-Handbuch (TTL, Flags, Fenstergröße) finden sich in Wireshark als benannte Zeilen wieder.

Kapitel 3: Installation und Laborumgebung

Lernziele

- Du installierst Wireshark unter Kali, Debian/Ubuntu und Fedora/Arch.
- Du richtest die nötigen Rechte für nicht-root-Mitschnitte ein.
- Du nutzt dasselbe isolierte Testnetz wie in den anderen Büchern dieser Reihe.

3.1 Installation

Beispiel 3.1: Wireshark installieren

```
# Kali Linux: bereits vorinstalliert
$ wireshark --version

# Debian/Ubuntu
$ sudo apt update && sudo apt install wireshark

# Fedora
$ sudo dnf install wireshark
# Arch Linux
$ sudo pacman -S wireshark-qt
```

Bei der Installation unter Debian/Ubuntu erscheint eine Rückfrage, ob Nicht-root-Benutzer:innen Mitschnitte erstellen dürfen – bestätige das, falls du nicht dauerhaft mit `sudo` arbeiten möchtest (siehe 3.2).

3.2 Rechte für Mitschnitte ohne root

Beispiel 3.2: Eigenen Benutzer zur Mitschnitt-Gruppe hinzufügen (Debian/Ubuntu)

```
$ sudo usermod -aG wireshark $USER
# Danach ab- und wieder anmelden, damit die Gruppenmitgliedschaft wirksam wird
```

Gut zu wissen

Dieses Prinzip kennst du bereits aus dem hping3-Handbuch (Kapitel 3.2): Statt dauerhaft mit erweiterten Rechten zu arbeiten, bekommt ein bestimmtes Programm (hier `dumpcap`, das eigentliche Mitschnitt-Werkzeug hinter Wireshark) gezielt die Capability `cap_net_raw` und `cap_net_admin`, meist automatisch über die Gruppenmitgliedschaft bei der Installation eingerichtet.

3.3 Grafische Oberfläche und Kommandozeile

Werkzeug	Charakter
`wireshark`	grafische Oberfläche, für explorative Analyse
`tshark`	Kommandozeilenversion mit denselben Filtern und Protokoll-Dissectoren
`dumpcap`	das schlanke, eigentliche Mitschnitt-Programm im Hintergrund beider Varianten

Dieses Buch zeigt Beispiele in **beiden** Formen: die grafische Oberfläche für die Analyse (Kapitel 4, 6–8), `tshark` für Automatisierung und Übungen, die sich gut reproduzieren lassen (Kapitel 9–10).

3.4 Das isolierte Testnetz

Dieses Buch nutzt dasselbe Labor wie die anderen Bände dieser Reihe: zwei virtuelle Maschinen in einem **Internal Network**, ohne Internetzugang.

Beispiel 3.3: Isolation vor jeder Übung verifizieren

```
$ ip route
# Keine Zeile mit "default via ..." -> kein bekannter Weg ins Internet
$ ping -c 2 192.168.56.102   # Zielsystem sollte erreichbar sein
```

LEGAL – nur mit Auftrag/Einwilligung

Starte einen Mitschnitt nur auf einer Schnittstelle, die ausschließlich Verkehr deines eigenen Testnetzes führt. Auf einem gemeinsam genutzten Schul-, Arbeits- oder Heim-WLAN würde ein Mitschnitt unweigerlich auch fremden Verkehr erfassen (Kapitel 1.4 und 2.2).

Zusammenfassung

Zusammenfassung

- Wireshark ist unter Kali vorinstalliert, auf anderen Distributionen leicht nachzuinstallieren.
- Mitschnitte ohne root-Rechte funktionieren über die Gruppenmitgliedschaft und die Capabilities von `dumpcap`.
- `wireshark` (grafisch) und `tshark` (Kommandozeile) nutzen dieselben Filter und Dissectoren.
- Dasselbe isolierte Internal-Network-Labor wie in den anderen Büchern dieser Reihe reicht für alle Übungen aus.

Kapitel 4: Die Oberfläche – Paketliste, Paketdetails, Bytes-Ansicht

Lernziele

- Du kannst einen Mitschnitt starten und gezielt beenden.
- Du kennst die drei zentralen Fensterbereiche der Wireshark-Oberfläche.
- Du kannst zwischen zusammengefasster und detaillierter Paketansicht wechseln.

4.1 Einen Mitschnitt starten

Beispiel 4.1: Mitschnitt über die Kommandozeile starten und stoppen

```
$ sudo tshark -i eth0 -w mitschnitt.pcapng
# Mit Strg+C beenden
```

In der grafischen Oberfläche genügt ein Doppelklick auf die gewünschte Schnittstelle im Startbildschirm; das rote Quadrat-Symbol in der Werkzeugleiste beendet den Mitschnitt wieder.

4.2 Die drei zentralen Fensterbereiche

Bereich	Inhalt
Paketliste (oben)	eine Zeile pro Paket: Nummer, Zeitstempel, Quelle, Ziel, Protokoll, Länge, Kurzinfo
Paketdetails (Mitte)	die OSI-geschichtete Aufschlüsselung des ausgewählten Pakets (Kapitel 2.3)
Bytes-Ansicht (unten)	die rohen Hexadezimal- und ASCII-Bytes des Pakets

Beispiel 4.2: Eine typische Zeile der Paketliste lesen

```
Nr.  Zeit      Quelle           Ziel             Protokoll  Länge  Info
42   1.203145  192.168.56.101   192.168.56.102   TCP   74
     51000 → 80 [SYN] Seq=0 Win=64240
```

Diese eine Zeile fasst zusammen, was im hping3-Handbuch als einzelner, manuell beobachteter Aufruf erscheint (hping3-Handbuch, Beispiel 4.2): Quelle, Ziel, Flags (`[SYN]`), Sequenznummer und Fenstergröße – nur eben automatisch für jedes mitgeschnittene Paket.

4.3 Zwischen den Ansichten navigieren

TIPP – Profi-Trick

Ein Klick auf eine Zeile der Paketliste synchronisiert automatisch Paketdetails und Bytes-Ansicht. Markierst du in den Paketdetails ein einzelnes Feld (z. B. die TTL), wird genau die zugehörige Byte-Position in der Bytes-Ansicht hervorgehoben – ein schneller Weg, um zu sehen, wo ein Feld tatsächlich im Paket steht.

Beispiel 4.3: Paketdetails eines TCP-Headers aufgeklappt (Auszug)

```
Transmission Control Protocol, Src Port: 51000, Dst Port: 80, Seq: 0, Len: 0
    Source Port: 51000
    Destination Port: 80
    Sequence Number: 0 (relative sequence number)
    Flags: 0x002 (SYN)
    Window: 64240
```

4.4 Farbregeln

Wireshark färbt Pakete standardmäßig nach Protokoll und Status ein (z. B. DNS meist blassblau, TCP-Fehler wie Retransmissions auffällig dunkel). Diese **Coloring Rules** lassen sich unter *Ansicht → Coloring Rules* anpassen.

> **Gut zu wissen**
>
> Die Standardfarbregeln sind ein guter erster Hinweis, ersetzen aber keine genaue Analyse: Ein rot/schwarz eingefärbtes Paket deutet oft auf ein Problem hin (z. B. eine Retransmission), ist aber kein automatischer Beweis für einen echten Fehler – gelegentliche Retransmissions kommen auch in gesunden Netzen vor.

Zusammenfassung

> **Zusammenfassung**
>
> - Ein Mitschnitt lässt sich grafisch per Doppelklick oder über `tshark -i <interface> -w <datei>` starten.
> - Paketliste, Paketdetails und Bytes-Ansicht sind die drei zentralen, miteinander synchronisierten Fensterbereiche.
> - Jede Zeile der Paketliste fasst zusammen, was im hping3-Handbuch als einzelne, manuelle Beobachtung erscheint.
> - Farbregeln geben erste Hinweise auf Auffälligkeiten, ersetzen aber keine genaue Prüfung.

Kapitel 5: Capture-Filter vs. Display-Filter

Lernziele

- Du kannst den Unterschied zwischen Capture- und Display-Filtern erklären.
- Du schreibst eigene Filterausdrücke für beide Filterarten.
- Du kombinierst Filter sinnvoll, um gezielt relevante Pakete zu finden.

5.1 Der entscheidende Unterschied

Dieser Unterschied ist einer der häufigsten Stolpersteine für Einsteiger:innen: Capture-Filter und Display-Filter sehen ähnlich aus, funktionieren aber grundlegend anders.

	Capture-Filter	Display-Filter
Wann wirksam	beim Mitschneiden selbst	nachträglich, auf bereits mitgeschnittene Pakete
Syntax	BPF (Berkeley Packet Filter)	Wireshark-eigene Display-Filter-Sprache
Nicht passende Pakete	werden gar nicht erst aufgezeichnet	bleiben im Mitschnitt, werden nur ausgeblendet
Beispiel	`tcp port 80`	`tcp.port == 80`

WARNUNG – Sicherheitsrisiko

Ein zu eng gesetzter **Capture-Filter** kann Pakete unwiederbringlich verloren gehen lassen, weil sie erst gar nicht aufgezeichnet wurden. Im Zweifel schneidest du lieber ohne oder mit großzügigem Capture-Filter mit und filterst anschließend mit einem **Display-Filter** gezielter nach – das lässt sich beliebig oft wiederholen, ohne den Mitschnitt zu wiederholen.

5.2 Capture-Filter (BPF-Syntax)

Beispiel 5.1: Typische Capture-Filter

```
host 192.168.56.102             # nur Verkehr von/zu diesem Host
port 80                         # nur Verkehr auf Port 80
tcp and port 80                 # nur TCP-Verkehr auf Port 80
not arp                         # ARP-Rauschen ausblenden
src host 192.168.56.101         # nur Verkehr, der von dieser Quelle kommt
```

Beispiel 5.2: Capture-Filter mit tshark anwenden

```
$ sudo tshark -i eth0 -f "tcp and port 80" -w http_mitschnitt.pcapng
```

5.3 Display-Filter (Wireshark-Syntax)

Beispiel 5.3: Typische Display-Filter

```
ip.addr == 192.168.56.102                # Verkehr von/zu dieser Adresse
tcp.port == 80                           # TCP-Verkehr auf Port 80
http                                     # nur HTTP-Pakete
tcp.flags.syn == 1 and tcp.flags.ack == 0 # nur reine SYN-Pakete
dns.qry.name contains "example"          # DNS-Anfragen nach einem Namensteil
tcp.analysis.retransmission              # nur erkannte Wiederholungsübertragungen
```

Beispiel 5.4: Display-Filter mit tshark anwenden

```
$ tshark -r mitschnitt.pcapng -Y "tcp.flags.syn == 1 and tcp.flags.ack == 0"
```

Gut zu wissen

`-f` bei `tshark` setzt einen Capture-Filter (BPF-Syntax), `-Y` einen Display-Filter (Wireshark-Syntax) – dieselbe Unterscheidung wie in der grafischen Oberfläche, nur über zwei verschiedene Optionen angesprochen. Verwechselst du beide, meldet `tshark` bei offensichtlich falscher Syntax einen Fehler, bei syntaktisch gültigen, aber semantisch falschen Ausdrücken bekommst du dagegen einfach kein oder ein unerwartetes Ergebnis.

5.4 Filter kombinieren

Beispiel 5.5: Mehrere Bedingungen kombinieren

```
ip.addr == 192.168.56.102 and tcp.port == 80 and tcp.flags.syn == 1
http.request.method == "POST" or http.request.method == "GET"
!(ip.addr == 192.168.56.1)   # alles außer dieser Adresse
```

TIPP – Profi-Trick

Die Autovervollständigung im Display-Filter-Feld der grafischen Oberfläche zeigt dir beim Tippen gültige Feldnamen an und färbt das Feld grün (gültig), rot (ungültig) oder gelb (gültig, aber ungewöhnlich) ein – eine der unterschätztesten Komfortfunktionen für den Einstieg.

Zusammenfassung

Zusammenfassung

- Capture-Filter (BPF-Syntax) entscheiden beim Mitschneiden, was überhaupt aufgezeichnet wird; verlorene Pakete sind unwiederbringlich weg.
- Display-Filter (Wireshark-Syntax) blenden nachträglich aus, was im Mitschnitt bereits vorhanden ist, und lassen sich beliebig anpassen.
- Im Zweifel großzügig mitschneiden, gezielt mit Display-Filtern nachfiltern.
- `tshark -f` setzt einen Capture-Filter, `tshark -Y` einen Display-Filter.

Kapitel 6: Protokolle erkennen – TCP, HTTP, DNS, ARP

Lernziele

- Du erkennst den TCP-Drei-Wege-Handshake in einem echten Mitschnitt.
- Du findest HTTP- und DNS-Anfragen samt Antworten.
- Du verstehst, wie ARP im lokalen Netz Adressen auflöst.

6.1 Den TCP-Drei-Wege-Handshake wiedererkennen

Beispiel 6.1: Drei-Wege-Handshake im Mitschnitt (Display-Filter: tcp.port == 80)

```
Nr.  Info
1    51000 → 80 [SYN] Seq=0 Win=64240
2    80 → 51000 [SYN, ACK] Seq=0 Ack=1 Win=65160
3    51000 → 80 [ACK] Seq=1 Ack=1 Win=64240
```

Das ist exakt derselbe Ablauf, den du im hping3-Handbuch (Kapitel 2.2) manuell Paket für Paket nachgebaut hast – hier siehst du ihn, wie er tatsächlich bei einem normalen Verbindungsaufbau abläuft, einschließlich der Zeitabstände zwischen den drei Paketen.

TIPP – Profi-Trick

Rechtsklick auf eines der drei Pakete → **Conversation Filter** → **TCP** zeigt dir ausschließlich den Verkehr genau dieser einen Verbindung, inklusive aller folgenden Datenpakete – praktischer als ein manuell getippter Display-Filter.

6.2 HTTP-Anfragen und -Antworten

Beispiel 6.2: HTTP-Anfrage im Klartext (Display-Filter: http.request)

```
GET /index.html HTTP/1.1
Host: labor.lokal
User-Agent: Mozilla/5.0
```

Beispiel 6.3: Zugehörige Antwort finden

```
Wireshark → Paket auswählen → Rechtsklick → "Follow" → "HTTP Stream"
```

Die Funktion **Follow HTTP Stream** fasst Anfrage und Antwort einer Interaktion zusammen in einem eigenen, gut lesbaren Fenster zusammen, statt sie über mehrere Pakete verteilt lesen zu müssen (vertieft in Kapitel 7).

6.3 DNS-Auflösung beobachten

Beispiel 6.4: DNS-Anfrage und -Antwort (Display-Filter: dns)

```
Nr.  Info
15   Standard query 0x1a2b A labor.lokal
16   Standard query response 0x1a2b A labor.lokal A 192.168.56.102
```

> **Gut zu wissen**
>
> DNS läuft standardmäßig über UDP Port 53. Bleibt die Antwort aus, kann das – wie bei UDP-Scans in hping3 und Nmap – mehrdeutig sein: ein nicht erreichbarer DNS-Server, ein Filter, oder ein verlorenes Paket. Wireshark zeigt dir allerdings zusätzlich, ob überhaupt eine Anfrage gesendet wurde, was die Fehlersuche gegenüber einem reinen Scan erleichtert.

6.4 ARP: Adressauflösung im lokalen Netz

Beispiel 6.5: ARP-Anfrage und -Antwort (Display-Filter: arp)

```
Nr.  Info
3    Who has 192.168.56.102? Tell 192.168.56.101
4    192.168.56.102 is at aa:bb:cc:dd:ee:ff
```

ARP löst eine IP-Adresse im lokalen Subnetz in die zugehörige MAC-Adresse auf – derselbe Mechanismus, der im hping3-Handbuch (Kapitel 2.1) als Grund genannt wird, warum lokale Scans oft zuverlässiger sind als Scans über mehrere Hops hinweg.

Zusammenfassung

> **Zusammenfassung**
>
> - Der TCP-Handshake lässt sich in einem Mitschnitt an drei aufeinanderfolgenden SYN-, SYN/ACK- und ACK-Paketen erkennen.
> - HTTP-Anfragen und -Antworten lassen sich einzeln lesen oder mit „Follow HTTP Stream" zusammengefasst betrachten.
> - DNS-Anfragen und -Antworten zeigen, ob und wie eine Namensauflösung stattgefunden hat.
> - ARP löst IP- in MAC-Adressen im lokalen Netz auf und erklärt, warum lokale Discovery so zuverlässig funktioniert.

Kapitel 7: TCP-Streams folgen und Klartext-Risiken erkennen

Lernziele

- Du nutzt "Follow TCP Stream", um eine ganze Verbindung zusammenhängend zu lesen.
- Du erkennst, warum unverschlüsselte Protokolle ein reales Risiko sind – am eigenen Mitschnitt, nicht nur in der Theorie.
- Du kennst die Gegenmaßnahme: verschlüsselte Protokolle und warum sie den Unterschied machen.

ILLEGAL – niemals ohne Auftrag

Dieses Kapitel zeigt, wie sich Zugangsdaten in unverschlüsseltem Verkehr erkennen lassen. Wende das **ausschließlich** auf Verkehr an, den du selbst in deinem isolierten Testnetz erzeugst (Kapitel 3.4 und 10), niemals auf fremden oder realen Verkehr. Das unbefugte Mitlesen fremder Zugangsdaten ist unabhängig vom technischen Weg strafbar (§ 202a/§ 202b StGB).

7.1 Eine ganze Verbindung zusammenhängend lesen

Beispiel 7.1: Einem TCP-Stream folgen

```
Paket auswählen → Rechtsklick → "Follow" → "TCP Stream"
```

Die Funktion setzt alle Pakete einer Verbindung in der richtigen Reihenfolge zu einem lesbaren Text zusammen, farblich nach Senderichtung unterschieden (üblicherweise rot für eine Richtung, blau für die andere) – deutlich angenehmer als das Mitlesen einzelner, verteilter Pakete.

7.2 Klartext-Zugangsdaten in einem eigenen Testaufbau

Beispiel 7.2: HTTP Basic Authentication im Klartext (eigener Testaufbau)

```
GET /admin HTTP/1.1
Host: labor.lokal
Authorization: Basic YWRtaW46cGFzc3dvcnQxMjM=
```

Gut zu wissen

Die Zeichenkette nach `Basic` ist lediglich **Base64-kodiert**, nicht verschlüsselt – eine Kodierung lässt sich ohne jeden Schlüssel in Sekunden zurückrechnen (`echo YWRtaW46cGFzc3dvcnQxMjM= | base64 -d` liefert `admin:passwort123`). Das ist der Kern des Problems: HTTP Basic Authentication über unverschlüsseltes HTTP schützt Zugangsdaten praktisch gar nicht vor jemandem, der mitschneiden kann.

Dasselbe Prinzip gilt für weitere klassische, unverschlüsselte Protokolle:

Protokoll	Risiko im Klartext	Sichere Alternative
HTTP (mit Basic/Form-Auth)	Zugangsdaten direkt lesbar oder leicht dekodierbar	HTTPS (TLS-verschlüsselt)
FTP	Benutzername und Passwort im Klartext	SFTP oder FTPS

Protokoll	Risiko im Klartext	Sichere Alternative
Telnet	gesamte Sitzung inkl. Zugangsdaten im Klartext	SSH
POP3/IMAP (ohne TLS)	Zugangsdaten im Klartext	POP3S/IMAPS

Beispiel 7.3: Telnet-Anmeldung im Klartext (eigener Testaufbau)

```
login: admin
password: geheim123
```

7.3 Warum Verschlüsselung den entscheidenden Unterschied macht

Beispiel 7.4: Derselbe Vorgang über eine verschlüsselte Verbindung (Konzept)

```
Display-Filter: tls
-> Die Paketdetails zeigen "Application Data" statt lesbarem Text.
   Ohne den passenden Sitzungsschlüssel bleibt der Inhalt für Wireshark
   unlesbar, selbst bei vollständigem Mitschnitt.
```

TIPP – Profi-Trick

Wiederhole denselben Testaufbau einmal über HTTP und einmal über HTTPS gegen dieselbe eigene Test-Webanwendung (Übung 5 in Kapitel 10). Der Unterschied zwischen lesbarem Klartext und undurchsichtigem „Application Data" ist eine der eindrücklichsten Demonstrationen dafür, warum TLS heute Standard sein sollte.

Zusammenfassung

Zusammenfassung

- "Follow TCP Stream" fasst eine ganze Verbindung zu einem zusammenhängenden, lesbaren Text zusammen.
- HTTP Basic Authentication ist lediglich Base64-kodiert, nicht verschlüsselt, und bietet praktisch keinen Schutz.
- HTTP, FTP, Telnet und unverschlüsseltes POP3/IMAP übertragen Zugangsdaten im Klartext; TLS-basierte Alternativen schützen davor.
- Jede Demonstration dieser Art gehört ausschließlich in den eigenen, isolierten Testaufbau.

Kapitel 8: Statistiken und Visualisierung

Lernziele

- Du nutzt die Protocol-Hierarchy-Ansicht für einen schnellen Überblick.
- Du analysierst Gesprächspartner und Datenmengen mit der Conversations-Ansicht.
- Du erkennst ungewöhnliche Verkehrsmuster in einem IO-Graphen.

8.1 Protocol Hierarchy: Was steckt überhaupt im Mitschnitt?

Beispiel 8.1: Protokollverteilung aufrufen

```
Statistics → Protocol Hierarchy
```

Beispiel 8.2: Typische Ausgabe (vereinfacht)

```
Protocol                Percent Packets  Packets  Bytes
Frame                   100.0            1200     980000
  Ethernet              100.0            1200     980000
    Internet Protocol   98.5             1182     965000
      TCP               76.2             915      820000
        HTTP            12.1             145      210000
      UDP               22.3             267      145000
        DNS             20.1             241      38000
```

Das ist oft der beste erste Schritt bei einem unbekannten Mitschnitt: ein schneller Überblick, welche Protokolle überhaupt vorkommen, bevor du gezielt filterst (Kapitel 5).

8.2 Conversations: Wer spricht mit wem?

Beispiel 8.3: Gesprächspartner analysieren

```
Statistics → Conversations → Reiter "TCP" oder "IPv4"
```

Die Conversations-Ansicht listet jede Verbindung mit Quelle, Ziel, Paketanzahl, übertragener Datenmenge und Dauer – ideal, um etwa den Host zu finden, der am meisten Datenverkehr erzeugt, oder eine ungewöhnlich lange Verbindung zu identifizieren.

TIPP – Profi-Trick

Ein Rechtsklick auf eine Zeile in der Conversations-Ansicht bietet „Apply as Filter" an und erzeugt automatisch den passenden Display-Filter für genau diese eine Verbindung – schneller als ihn von Hand zu tippen.

8.3 IO-Graph: Verkehr über die Zeit

Beispiel 8.4: Verkehrsverlauf visualisieren

```
Statistics → I/O Graph
```

Der IO-Graph zeigt Paketrate oder Datenmenge über die Zeit als Diagramm. Ein plötzlicher, steiler Anstieg kann auf einen Portscan, einen Datentransfer oder – wie im hping3-Handbuch (Kapitel 8) beschrieben – auf einen SYN-Flood-Test im eigenen Labor hindeuten.

Gut zu wissen

Du kannst mehrere Filterausdrücke gleichzeitig als separate Linien im selben IO-Graphen darstellen, zum Beispiel `tcp.flags.syn == 1` neben dem Gesamtverkehr – so siehst du auf einen Blick, ob ein beobachteter Anstieg tatsächlich an ungewöhnlich vielen SYN-Paketen liegt.

Zusammenfassung

Zusammenfassung

- Protocol Hierarchy liefert einen schnellen Überblick über alle im Mitschnitt enthaltenen Protokolle.
- Conversations zeigt Gesprächspartner, Datenmengen und Verbindungsdauer.
- Der IO-Graph macht Verkehrsmuster über die Zeit sichtbar, etwa Lastspitzen durch Scans oder Floods.
- Mehrere Filterlinien im selben Graphen lassen sich gezielt vergleichen.

Kapitel 9: Mitschnitte speichern, exportieren und mit tshark automatisieren

Lernziele

- Du speicherst und filterst Mitschnitte im passenden Format.
- Du exportierst einzelne Objekte (z. B. übertragene Dateien) aus einem Mitschnitt.
- Du automatisierst wiederkehrende Analysen mit tshark.

9.1 Speicherformate

Endung	Format	Hinweis
`.pcapng`	Standardformat moderner Wireshark-Versionen	enthält mehr Metadaten als das ältere `.pcap`
`.pcap`	älteres, weiter verbreitetes Format	von praktisch jedem Analysewerkzeug lesbar

Beispiel 9.1: Nur gefilterte Pakete in eine neue Datei speichern

```
Datei → Export Specified Packets → "Displayed" statt "All packets" wählen
```

So entsteht aus einem großen Mitschnitt eine kleine, auf das Wesentliche reduzierte Datei – nützlich, um Ergebnisse weiterzugeben, ohne den gesamten, möglicherweise sensiblen Mitschnitt zu teilen (siehe Kapitel 1.4).

9.2 Objekte exportieren

Beispiel 9.2: Übertragene Dateien aus einem HTTP-Mitschnitt extrahieren

```
Datei → Export Objects → HTTP
```

Diese Funktion listet alle über HTTP übertragenen Dateien (Bilder, Dokumente, Skripte) im Mitschnitt auf und erlaubt, sie einzeln zu speichern – praktisch für Nachweise im Rahmen eines autorisierten Tests (Ethical-Hacking-Handbuch, Kapitel 8).

9.3 Automatisierung mit tshark

Beispiel 9.3: Wiederholte Analyse mit tshark skripten

```
$ tshark -r mitschnitt.pcapng -Y "http.request" -T fields \
    -e ip.src -e http.host -e http.request.uri
192.168.56.101  labor.lokal  /index.html
192.168.56.101  labor.lokal  /admin
```

`-T fields` zusammen mit `-e <feldname>` gibt gezielt einzelne Felder aus, statt der vollständigen Paketdarstellung – ideal, um Ergebnisse in eine Tabelle oder ein weiteres Skript einzuspeisen.

Beispiel 9.4: Statistik direkt über die Kommandozeile

```
$ tshark -r mitschnitt.pcapng -q -z io,phs   # Protocol Hierarchy als Text
$ tshark -r mitschnitt.pcapng -q -z conv,tcp # TCP-Conversations als Text
```

TIPP – Profi-Trick

`tshark` eignet sich hervorragend, um eine Analyse, die du in der grafischen Oberfläche einmal manuell entwickelt hast, anschließend zu automatisieren – etwa als wiederkehrende Prüfung nach jedem Testlauf im eigenen Labor.

Zusammenfassung

Zusammenfassung

- `.pcapng` ist das moderne Standardformat, `.pcap` das ältere, breiter kompatible.
- Export Specified Packets und Export Objects erlauben, gezielt Teilmengen eines Mitschnitts weiterzugeben.
- `tshark` mit `-T fields`/`-e` extrahiert gezielt einzelne Werte für die Weiterverarbeitung.
- Statistiken wie Protocol Hierarchy und Conversations lassen sich auch direkt über die Kommandozeile abrufen.

Kapitel 10: Praxisteil – Laboraufbau und geführte Übungen

Lernziele

- Du baust das für dieses Buch nötige Labor auf.
- Du arbeitest acht Übungen durch, die alle vorherigen Kapitel praktisch verbinden.
- Du weißt, dass abweichende Ergebnisse oft an Netzwerkkonfiguration oder Timing liegen, nicht automatisch an einem Fehler.

10.1 Laboraufbau

Dieses Buch nutzt dasselbe Labor-Grundgerüst wie die anderen Bände dieser Reihe: Kali-VM (mit Wireshark, hping3 und Nmap) und Metasploitable-2-VM im selben Internal Network, ohne Internetzugang (Kapitel 3).

10.2 Checkliste: sichere Laborumgebung

Checkliste 10.1: Vor jeder Übung

```
[ ] Netzwerkadapter aller beteiligten VMs steht auf "Internes Netzwerk"
[ ] `ip route` zeigt keine Standardroute ins Internet
[ ] Mitschnitt läuft ausschließlich auf einer Schnittstelle des eigenen Testnetzes
[ ] Für Übung 5/6: eigene, dafür vorgesehene Testanwendung mit Anmeldefunktion bereit
```

10.3 Übungen

Jede Übung folgt demselben Schema: Ziel, Voraussetzungen, Durchführung, erwartetes Ergebnis, Auswertung und Fehlerbehebung.

Übung 1: Ersten Mitschnitt starten und lesen

Ziel: Die drei zentralen Fensterbereiche der Oberfläche kennenlernen. **Voraussetzungen:** Kali-VM mit Wireshark, erreichbare Metasploitable-VM. **Durchführung:** Starte einen Mitschnitt auf der internen Schnittstelle, rufe von einem Terminal aus `ping -c 3 192.168.56.102` auf, stoppe den Mitschnitt danach. **Erwartetes Ergebnis:** Mehrere ICMP-Pakete erscheinen in der Paketliste, paarweise Anfrage und Antwort. **Auswertung:** Klicke ein Anfrage-Paket an und vergleiche die OSI-Schichten in den Paketdetails mit Kapitel 2.3. **Fehlerbehebung:** Erscheinen keine Pakete, prüfe, ob die richtige Schnittstelle ausgewählt wurde (Kapitel 2.1).

Übung 2: Den TCP-Handshake beobachten

Ziel: Den Drei-Wege-Handshake in einem echten Mitschnitt identifizieren. **Voraussetzungen:** wie Übung 1. **Durchführung:** Starte einen Mitschnitt, führe `curl http://192.168.56.102/` aus, stoppe den Mitschnitt, filtere mit `tcp.port == 80` (Beispiel 6.1). **Erwartetes Ergebnis:** Die ersten drei Pakete zeigen SYN, SYN/ACK, ACK in dieser Reihenfolge. **Auswertung:** Vergleiche Sequenznummern und Fenstergröße mit dem, was du im hping3-Handbuch (Kapitel 2.2) über diese Felder gelernt hast. **Fehlerbehebung:** Siehst du mehr als drei Pakete am Anfang, prüfe, ob weitere Hintergrundverbindungen (z. B. DNS vor dem HTTP-Aufruf) mitgeschnitten wurden – filtere zusätzlich nach der Ziel-IP.

Übung 3: Capture-Filter vs. Display-Filter vergleichen

Ziel: Den Unterschied zwischen beiden Filterarten praktisch erleben. **Voraussetzungen:** wie Übung 1. **Durchführung:** Starte einen Mitschnitt mit dem Capture-Filter `tcp and port 80` (Beispiel 5.2), erzeuge gemischten Verkehr (HTTP-Aufruf und `ping`). Stoppe den Mitschnitt und versuche danach, die ICMP-Pakete mit einem Display-Filter zu finden. **Erwartetes Ergebnis:** Die ICMP-Pakete fehlen vollständig, weil der Capture-Filter sie nie aufgezeichnet hat. **Auswertung:** Das zeigt den zentralen Unterschied aus Kapitel 5.1: Ein Display-Filter kann nur anzeigen, was der Capture-Filter überhaupt zugelassen hat. **Fehlerbehebung:** Erscheinen doch ICMP-Pakete, prüfe, ob der Capture-Filter beim Start tatsächlich korrekt übernommen wurde.

Übung 4: Protocol Hierarchy und Conversations nutzen

Ziel: Einen unbekannten, etwas größeren Mitschnitt systematisch erschließen. **Voraussetzungen:** ein Mitschnitt mit gemischtem Verkehr (z. B. aus Übung 3, ohne Capture-Filter wiederholt). **Durchführung:** Öffne Statistics → Protocol Hierarchy, danach Statistics → Conversations (Beispiel 8.1–8.3). **Erwartetes Ergebnis:** Eine Übersicht der enthaltenen Protokolle sowie eine Liste der Verbindungen mit Datenmengen. **Auswertung:** Identifiziere die Verbindung mit der größten übertragenen Datenmenge und erkläre dir, warum sie entstanden ist. **Fehlerbehebung:** Ist die Liste leer, prüfe, ob tatsächlich Verkehr während des Mitschnitts erzeugt wurde.

Übung 5: Klartext-Zugangsdaten in der eigenen Testanwendung erkennen

Ziel: Das Risiko unverschlüsselter Authentifizierung selbst nachvollziehen. **Voraussetzungen:** eine eigene, einfache Testanwendung mit HTTP-Basic-Auth oder Formular-Login (z. B. DVWA aus dem Ethical-Hacking-Handbuch), **keine echten Zugangsdaten**. **Durchführung:** Starte einen Mitschnitt, melde dich mit einem frei erfundenen Test-Benutzernamen und -Passwort an, stoppe den Mitschnitt, nutze „Follow HTTP Stream" (Beispiel 7.1–7.2). **Erwartetes Ergebnis:** Benutzername und Passwort erscheinen lesbar oder leicht dekodierbar im Stream. **Auswertung:** Notiere, an welcher Stelle genau die Zugangsdaten sichtbar werden. **Fehlerbehebung:** Erscheinen die Daten verschlüsselt (`Application Data`), läuft die Testanwendung bereits über HTTPS – wiederhole den Test mit einer bewusst unverschlüsselten Variante.

Übung 6: Denselben Login über HTTPS vergleichen

Ziel: Den Unterschied zwischen Klartext und Verschlüsselung direkt sehen. **Voraussetzungen:** dieselbe Testanwendung, zusätzlich mit HTTPS erreichbar. **Durchführung:** Wiederhole Übung 5 über die HTTPS-Adresse derselben Anwendung. **Erwartetes Ergebnis:** Im Stream erscheint „Application Data" statt lesbarem Text. **Auswertung:** Vergleiche beide Mitschnitte nebeneinander – das ist der praktische Nachweis aus Kapitel 7.3. **Fehlerbehebung:** Ist die Testanwendung nicht über HTTPS erreichbar, richte testweise ein selbstsigniertes Zertifikat ein (zahlreiche Anleitungen für DVWA/Apache verfügbar) oder nutze ersatzweise eine andere, TLS-fähige Testanwendung.

Übung 7: Einen Nmap-Scan im Mitschnitt wiedererkennen

Ziel: Die Spuren eines automatisierten Scans im Rohverkehr identifizieren. **Voraussetzungen:** Nmap installiert (Nmap-Handbuch, Kapitel 3). **Durchführung:** Starte einen Mitschnitt, führe `sudo nmap -sS -p 1-100 192.168.56.102` aus (Nmap-Handbuch, Beispiel 4.1), stoppe den Mitschnitt und nutze den IO-Graphen (Beispiel 8.4). **Erwartetes Ergebnis:** Eine kurze, deutlich erkennbare Spitze vieler SYN-Pakete in kurzer Zeit. **Auswertung:** Filtere zusätzlich mit `tcp.flags.syn == 1 and tcp.flags.ack == 0` und vergleiche die Anzahl mit der gescannten Portzahl. **Fehlerbehebung:** Siehst du deutlich weniger SYN-Pakete als erwartet, prüfe, ob der Mitschnitt rechtzeitig vor dem Scan gestartet wurde.

Übung 8: tshark-Automatisierung

Ziel: Eine wiederkehrende Auswertung ohne grafische Oberfläche durchführen. **Voraussetzungen:** ein gespeicherter Mitschnitt aus einer der vorherigen Übungen. **Durchführung:** Führe `tshark -r <datei> -Y "http.request" -T fields -e ip.src -e http.host -e http.request.uri` aus (Beispiel 9.3). **Erwartetes Ergebnis:** Eine Tabelle aller HTTP-Anfragen mit Quelladresse, Host und Pfad. **Auswertung:** Vergleiche die Ausgabe mit dem, was du zuvor in der grafischen Oberfläche für dieselben Pakete gesehen hast. **Fehlerbehebung:** Bleibt die Ausgabe leer, prüfe mit `tshark -r <datei> -Y http` ohne Feldbeschränkung, ob überhaupt HTTP-Pakete im Mitschnitt enthalten sind.

Zusammenfassung

Zusammenfassung

- Dasselbe Internal-Network-Labor wie in den anderen Büchern dieser Reihe reicht für alle acht Übungen aus.
- Die Übungen verbinden Grundbedienung, Filter, Protokollerkennung, Klartext-Risiken, Statistiken und Automatisierung.
- Mehrere Übungen verweisen bewusst auf das hping3- und Nmap-Handbuch, um dieselben Vorgänge aus Erzeuger- und Beobachterperspektive zu vergleichen.

Kapitel 11: Troubleshooting, Spickzettel, Glossar und Ressourcen

Lernziele

- Du kannst die häufigsten Wireshark-Probleme einordnen und beheben.
- Du hast einen kompakten Spickzettel für den Alltag zur Hand.
- Du kennst weiterführende Ressourcen für die Vertiefung.

11.1 Häufige Probleme

Meldung / Symptom	Wahrscheinliche Ursache	Lösung
Keine Schnittstellen in der Liste wählbar	fehlende Rechte für Mitschnitte	Gruppenmitgliedschaft prüfen (Kapitel 3.2) oder mit `sudo` starten
Mitschnitt bleibt leer	falsche Schnittstelle, oder Capture-Filter zu eng	Schnittstelle prüfen, Capture-Filter testweise entfernen
Display-Filter-Feld bleibt rot	ungültige Syntax	Autovervollständigung nutzen, Feldnamen mit `tshark -G fields` nachschlagen
"Follow TCP Stream" zeigt nichts Sinnvolles	verschlüsselte Verbindung (TLS) ohne Schlüssel	erwartetes Verhalten bei HTTPS ohne Schlüsselmaterial (Kapitel 7.3)
Sehr große, unübersichtliche Mitschnitte	zu lange oder zu breit mitgeschnitten	gezielter Capture-Filter, oder nachträglich mit Export Specified Packets eingrenzen
Zeitstempel wirken unplausibel	falsche Zeitzone oder fehlerhafte Systemuhr der VM	Systemzeit der VM prüfen, Zeitanzeige in Wireshark auf „UTC“ oder „Lokal“ umstellen

11.2 Spickzettel

Beispiel 11.1: Die wichtigsten Wireshark-/tshark-Befehle auf einen Blick

```
# Verfügbare Schnittstellen auflisten
tshark -D

# Mitschnitt starten und in Datei speichern
sudo tshark -i eth0 -w mitschnitt.pcapng

# Mit Capture-Filter mitschneiden
sudo tshark -i eth0 -f "tcp and port 80" -w http.pcapng
# Gespeicherten Mitschnitt mit Display-Filter lesen
tshark -r mitschnitt.pcapng -Y "tcp.flags.syn == 1"

# Nur bestimmte Felder ausgeben
tshark -r mitschnitt.pcapng -T fields -e ip.src -e ip.dst -e tcp.port

# Protocol Hierarchy / Conversations über die Kommandozeile
tshark -r mitschnitt.pcapng -q -z io,phs
tshark -r mitschnitt.pcapng -q -z conv,tcp
```

11.3 Glossar

- **BPF (Berkeley Packet Filter)** – die Filtersprache, in der Capture-Filter geschrieben werden.
- **Capture-Filter** – ein beim Mitschneiden wirksamer Filter; nicht passende Pakete werden nie aufgezeichnet.
- **Conversation** – eine zusammenhängende Verbindung zwischen zwei Endpunkten, in Wireshark als eigene Ansicht auswertbar.
- **Display-Filter** – ein nachträglich auf bereits mitgeschnittene Pakete angewendeter Filter.
- **Dissector** – der Teil von Wireshark, der die Bytes eines Protokolls in lesbare Felder zerlegt.
- **dumpcap** – das schlanke Mitschnitt-Programm im Hintergrund von Wireshark und tshark.
- **pcap/pcapng** – die gängigen Dateiformate für gespeicherte Mitschnitte.
- **Promiscuous Mode** – ein Netzwerkkarten-Modus, der auch nicht an die eigene Adresse gerichtete Pakete annimmt.
- **tshark** – die Kommandozeilenversion von Wireshark mit identischen Filtern und Dissectoren.

11.4 Weiterführende Ressourcen

Ressource	Inhalt
Offizielles Wireshark User's Guide (wireshark.org)	vollständige, verbindliche Referenz
hping3-Handbuch (dieselbe Buchreihe)	manuelle Paketerzeugung als Gegenstück zur passiven Analyse
Nmap-Handbuch (dieselbe Buchreihe)	automatisierte Netzwerkerkundung, deren Verkehr sich mit Wireshark beobachten lässt
Ethical-Hacking-Handbuch (dieselbe Buchreihe)	Gesamtzusammenhang: Recon, Scanning, Exploitation, Reporting
Wireshark Sample Captures (wiki.wireshark.org)	offizielle Beispiel-Mitschnitte zum Üben ohne eigenes Labor
TryHackMe / HackTheBox	legale Übungsplattformen mit eigenen Paketanalyse-Räumen

TIPP – Profi-Trick

Die offiziellen **Sample Captures** im Wireshark-Wiki sind eine hervorragende Ergänzung zu den Übungen in diesem Buch: fertige, dokumentierte Mitschnitte zu bestimmten Protokollen oder Szenarien, mit denen du filtern und analysieren üben kannst, ohne selbst Verkehr erzeugen zu müssen.

Zusammenfassung

Zusammenfassung

- Die häufigsten Probleme betreffen fehlende Rechte, zu enge Capture-Filter oder fehlendes Schlüsselmaterial bei TLS.
- Der Spickzettel deckt die sechs häufigsten tshark-Befehlsmuster des gesamten Buches ab.
- Das offizielle User's Guide, die Sample Captures sowie das hping3- und Nmap-Handbuch sind die wichtigsten Anschlussstellen zum Weiterlernen.

Verzeichnis der Befehlsbeispiele

Beispiel 1.1: Typischer Ablauf einer Paketanalyse (Konzept) … Seite 8
Beispiel 2.1: Verfügbare Schnittstellen auflisten … Seite 9
Beispiel 2.2: Promiscuous Mode beim Start eines Mitschnitts … Seite 9
Beispiel 2.3: Die Schichten eines einzelnen Pakets (Konzept) … Seite 10
Beispiel 3.1: Wireshark installieren … Seite 11
Beispiel 3.2: Eigenen Benutzer zur Mitschnitt-Gruppe hinzufügen (Debian/Ubuntu) … Seite 11
Beispiel 3.3: Isolation vor jeder Übung verifizieren … Seite 12
Beispiel 4.1: Mitschnitt über die Kommandozeile starten und stoppen … Seite 13
Beispiel 4.2: Eine typische Zeile der Paketliste lesen … Seite 13
Beispiel 4.3: Paketdetails eines TCP-Headers aufgeklappt (Auszug) … Seite 14
Beispiel 5.1: Typische Capture-Filter … Seite 15
Beispiel 5.2: Capture-Filter mit tshark anwenden … Seite 15
Beispiel 5.3: Typische Display-Filter … Seite 16
Beispiel 5.4: Display-Filter mit tshark anwenden … Seite 16
Beispiel 5.5: Mehrere Bedingungen kombinieren … Seite 16
Beispiel 6.1: Drei-Wege-Handshake im Mitschnitt (Display-Filter: tcp.port == 80) … Seite 17
Beispiel 6.2: HTTP-Anfrage im Klartext (Display-Filter: http.request) … Seite 17
Beispiel 6.3: Zugehörige Antwort finden … Seite 17
Beispiel 6.4: DNS-Anfrage und -Antwort (Display-Filter: dns) … Seite 18
Beispiel 6.5: ARP-Anfrage und -Antwort (Display-Filter: arp) … Seite 18
Beispiel 7.1: Einem TCP-Stream folgen … Seite 19
Beispiel 7.2: HTTP Basic Authentication im Klartext (eigener Testaufbau) … Seite 19
Beispiel 7.3: Telnet-Anmeldung im Klartext (eigener Testaufbau) … Seite 20
Beispiel 7.4: Derselbe Vorgang über eine verschlüsselte Verbindung (Konzept) … Seite 20
Beispiel 8.1: Protokollverteilung aufrufen … Seite 21
Beispiel 8.2: Typische Ausgabe (vereinfacht) … Seite 21
Beispiel 8.3: Gesprächspartner analysieren … Seite 21
Beispiel 8.4: Verkehrsverlauf visualisieren … Seite 21
Beispiel 9.1: Nur gefilterte Pakete in eine neue Datei speichern … Seite 23
Beispiel 9.2: Übertragene Dateien aus einem HTTP-Mitschnitt extrahieren … Seite 23
Beispiel 9.3: Wiederholte Analyse mit tshark skripten … Seite 23
Beispiel 9.4: Statistik direkt über die Kommandozeile … Seite 24
Checkliste 10.1: Vor jeder Übung … Seite 25
Beispiel 11.1: Die wichtigsten Wireshark-/tshark-Befehle auf einen Blick … Seite 28

Index

ARP – 18
BPF – 15, 29
Capture-Filter – 15, 26, 29
Conversation – 21, 26, 29
Display-Filter – 15, 16, 17, 18, 26, 29
Dissector – 29
DNS – 7, 14, 18, 26
dumpcap – 29
HTTP – 17, 19, 23, 26, 27
IO-Graph – 21, 22, 27
OSI-Modell – 10
pcap – 29
pcapng – 29
Promiscuous Mode – 9, 29
Protocol Hierarchy – 21, 26
TLS – 7, 26
tshark – 12, 15, 16, 23, 27, 28, 29
Wireshark – 5, 7, 9, 10, 11, 14, 16, 25

WIRESHARK DAS PRAXISHANDBUCH

Was wirklich über die Leitung geht

Wireshark ist der Gegenpol zu hping3 und Nmap: Statt Pakete zu erzeugen, macht es sichtbar, was im Netzwerk tatsächlich passiert. Dieses Handbuch führt von der Schnittstellenwahl über Capture- und Display-Filter bis zur Protokollanalyse, zeigt anhand eigener Testaufbauten, warum unverschlüsselte Protokolle ein reales Risiko sind, und verbindet die Erzeuger-Perspektive der beiden anderen Bände mit der Beobachter-Perspektive der Paketanalyse.

DAS ERWARTET DICH

- Schnittstellen, Promiscuous Mode und das OSI-Modell in der Praxis
- Capture-Filter vs. Display-Filter – der wichtigste Unterschied für Einsteiger:innen
- TCP-Streams folgen und Klartext-Zugangsdaten im eigenen Testaufbau erkennen
- Statistiken: Protocol Hierarchy, Conversations und IO-Graphen
- tshark-Automatisierung, 8 geführte Übungen, Spickzettel und Glossar

Für wen?

Leser:innen des Ethical-Hacking-, hping3- und Nmap-Handbuchs, die die Analyseseite der Netzwerksicherheit vertiefen möchten, sowie Studierende und Auszubildende. Ausschließlich für legale, autorisierte Mitschnitte und eigene Laborumgebungen.

31 Seiten · Stand Oktober 2026 · Ergänzung zum Ethical-Hacking-, hping3- und Nmap-Handbuch

Hinweis: Keine Rechts- oder Anlageberatung. Marken- und Produktnamen gehören ihren jeweiligen Inhabern.